МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторные работы
по курсу «Информационный поиск»

Выполнил: Шелаев Серафим Ильич
Группа: М8О-408Б-22
Преподаватель: Кухтичев Антон Алексеевич

Москва, 2026

Содержание

1 Введение **3**

2 Лабораторная работа №1: Добыча корпуса документов **3**
2.1 Цель работы . . . 3
2.2 Описание источников . . . 3
2.3 Структура документа . . . 3
2.4 Выделение текста . . . 3
2.5 Статистика корпуса . . . 4
2.6 Проверка пригодности . . . 4

3 Лабораторная работа №2: Поисковый робот **4**
3.1 Цель работы . . . 4
3.2 Архитектура и стек технологий . . . 4
3.3 Алгоритм работы . . . 4
3.4 Результат . . . 5

4 Лабораторная работа №3: Токенизация **5**
4.1 Цель работы . . . 5
4.2 Правила токенизации . . . 5
4.3 Достоинства метода . . . 5
4.4 Недостатки метода . . . 5
4.5 Примеры неудачной токенизации . . . 6
4.6 Статистика токенизации . . . 6
4.7 Производительность . . . 6

5 Лабораторная работа №4: Стемминг **6**
5.1 Цель работы . . . 6
5.2 Метод решения . . . 7
5.3 Результаты . . . 7
5.4 Интеграция в поисковую систему . . . 7

6 Лабораторная работа №5: Закон Ципфа **7**
6.1 Цель работы . . . 7
6.2 Методика . . . 8
6.3 Результаты . . . 8
6.4 Причины расхождений . . . 9
6.5 Выводы . . . 9

7 Лабораторная работа №6: Булев индекс **9**
7.1 Цель работы . . . 9
7.2 Внутреннее представление документа . . . 9
7.3 Реализация индексатора . . . 9
7.3.1 Структуры данных . . . 10
7.3.2 Метод сортировки . . . 10
7.3.3 Бинарный формат индекса (побайтовое описание) . . . 10
7.4 Результаты . . . 11
7.5 Скорость индексации . . . 11
7.6 Анализ оптимальности . . . 11

8 Лабораторная работа №7: Булев поиск **12**
8.1 Цель работы 12
8.2 Синтаксис запросов 12
8.3 Алгоритм поиска 12
8.4 Примеры запросов 13
8.5 Веб-интерфейс 13
8.6 CLI-утилита 13
8.7 Производительность 13
8.8 Тестирование корректности 13

9 Лабораторная работа №8: Ранжирование TF-IDF **14**
9.1 Цель работы 14
9.2 Формулы 14
9.3 Интерпретация запросов 14
9.4 Примеры запросов 14

10 Выводы **15**

1 Введение

В рамках курса «Информационный поиск» была разработана полнофункциональная поисковая система **SportSearch**, включающая подсистемы сбора данных, обработки текста (токенизация, стемминг), индексации, булева и ранжированного поиска. В качестве корпуса использованы статьи англоязычной Википедии по тематике «Спорт».

Ядро системы (индексация и поисковый движок) реализовано на языке C++ с использованием собственных структур данных:

- **DynArray** — динамический массив (замена `std::vector`).
- **HashDict** — хеш-таблица с открытой адресацией и хеш-функцией.

Обвязка (скачивание статей, импорт в БД, веб-интерфейс) написана на Python. Хранение метаданных — MongoDB. Веб-интерфейс — Flask.

2 Лабораторная работа №1: Добыча корпуса документов

2.1 Цель работы

Собрать и проанализировать корпус текстовых документов объёмом не менее 30 000 статей для последующей индексации и поиска.

2.2 Описание источников

В качестве предметной области выбран **спорт**. Источник данных — англоязычная **Википедия**. Статьи скачиваются через Wikipedia REST API по 40 тематическим запросам: `football`, `basketball`, `tennis`, `Olympic Games`, `FIFA World Cup`, `ice hockey`, `swimming sport`, `boxing`, `Formula One`, `rugby union`, `cycling sport`, `golf sport`, `athletics`, `gymnastics`, `wrestling sport`, `esports` и др.

Документы представляют собой энциклопедические статьи, содержащие заголовок и основной текст. Язык документов — английский.

2.3 Структура документа

«Сырой» документ (JSON от API) содержит:

- Заголовок статьи (`title`).
- Полный текст в виде HTML (`extract`).
- URL страницы.

2.4 Выделение текста

Для выделения текста используется парсер, удаляющий HTML-теги, служебные блоки, таблицы и навигационные элементы. В итоговый текстовый документ включаются:

1. Заголовок.
2. Основной текст статьи (очищенный от разметки).

Результат сохраняется в каталог `data/text/wikipedia/` — по одному `.txt`-файлу на статью. Исходные данные (JSON) сохраняются в `data/raw/wikipedia/`.

2.5 Статистика корпуса

Параметр	Значение
Количество документов	31016
Средний размер текста (на документ)	12000 байт
Общий объём выделенного текста	340 МБ

Таблица 1: Статистика собранного корпуса

2.6 Проверка пригодности

Поиск по Википедии доступен через встроенный поиск (`https://en.wikipedia.org/w/index.php?search=...`), а также через Google с ограничением `site:en.wikipedia.org`. Это позволяет использовать данные источники как альтернативы для сравнительной оценки качества поиска.

Примеры запросов к существующим поисковикам:

- `FIFA World Cup history` — встроенный поиск Wikipedia находит основную статью, но не все связанные (отборочные турниры, история отдельных чемпионатов).
- `olympic gold medal records` — Google `site:en.wikipedia.org` выдаёт релевантные результаты, но встроенный поиск Википедии может путать с неспортивными значениями слова «record».

3 Лабораторная работа №2: Поисковый робот

3.1 Цель работы

Разработать поисковый робот (crawler) для автоматического сбора документов с поддержкой докачки и соблюдения вежливости.

3.2 Архитектура и стек технологий

Робот реализован на языке **Python**. Основные компоненты:

- **YAML-конфигурация** (`conf.yaml`): параметры запуска (задержки, timeout, user-agent, список seed-запросов).
- **MongoDB**: хранение метаданных документов и очереди скачивания.
- **Requests**: HTTP-запросы с повторными попытками (retries).
- **Wikipedia API**: скачивание статей через `action=query` с параметром `extracts`.

3.3 Алгоритм работы

1. **Инициализация**: чтение конфигурации, подключение к MongoDB.
2. **Поиск статей**: для каждого seed-запроса выполняется поиск через Wikipedia API (`list=search`). Найденные заголовки добавляются в очередь.
3. **Загрузка**: для каждого заголовка запрашивается полный текст (`prop=extracts`). Между запросами выдерживается задержка (400 мс).

4. **Дедупликация**: хеш SHA-256 используется для обнаружения дубликатов; повторно скачанные страницы обновляют метку `fetched_at`.

5. **Докачка**: при перезапуске робот продолжает с того места, где остановился, выбирая из базы ещё не скачанные URL.

3.4 Результат

Робот успешно собрал корпус спортивных статей из Википедии. Реализована устойчивость к разрывам соединения и возможность остановки/продолжения без потери прогресса.

4 Лабораторная работа №3: Токенизация

4.1 Цель работы

Реализовать разбиение текста на токены (слова) для построения поискового индекса.

4.2 Правила токенизации

Токенизатор реализован на C++ (файл `porter.hpp`, пространство имён `Porter`):

1. **Разбиение**: текст сканируется посимвольно; токеном считается непрерывная последовательность алфавитно-цифровых символов (`isalnum`).

2. **Нормализация**: все символы приводятся к нижнему регистру (`tolower`).

3. **Стемминг**: к каждому токену применяется алгоритм Портера (см. ЛР4).

4.3 Достоинства метода

- **Простота**: минимальные требования к памяти и вычислениям.
- **Высокая скорость**: однопроходный алгоритм $O(n)$.
- **Универсальность**: работает с любым текстом на латинице.
- **Унификация словоформ**: стемминг объединяет различные формы слова (players, playing, played $\rightarrow$ play).

4.4 Недостатки метода

- **Потеря информации**: агрессивный стемминг может приводить к коллизиям (например, `university` $\rightarrow$ `univers`, при этом `universal` $\rightarrow$ `univers`).
- **Разбиение составных терминов**: дефисы являются разделителями, поэтому `100-meter` становится двумя токенами `100` и `meter`.
- **Числа**: даты и счёт матчей (`2-1`, `2024`) попадают в словарь как отдельные числовые токены.

4.5 Примеры неудачной токенизации

Исходный текст	**Результат**	**Проблема**
100-meter	100, meter	Разбито на два токена
university	univers	Коллизия с «universal»
2–1	2, 1	Счёт матча потерян
FIFA's	fifa, s	Апостроф разделяет слово

Таблица 2: Примеры неудачной токенизации

Возможные улучшения:

- Добавить словарь составных терминов (n-grams) для спортивной лексики.
- Обрабатывать апострофы (`'s`, `'t`) как часть слова.
- Применить лемматизацию вместо стемминга для снижения коллизий.

4.6 Статистика токенизации

Параметр	**Значение**
Количество документов	31016
Общее количество токенов	61094782
Уникальных токенов (до стемминга)	680753
Среднее токенов на документ	8227

Таблица 3: Статистика токенизации

4.7 Производительность

Параметр	**Значение**
Время токенизации (1000 файлов)	1,52 сек
Скорость токенизации	8,32 КБ/сек

Таблица 4: Производительность токенизатора

Зависимость от объёма: время линейно зависит от размера входных данных ($T = O(n)$), так как алгоритм однопроходный. Возможные способы ускорения: многопоточная обработка, SIMD-инструкции, memory-mapped I/O.

5 Лабораторная работа №4: Стемминг

5.1 Цель работы

Реализовать морфологическую нормализацию (стемминг) для улучшения полноты поиска.

5.2 Метод решения

Реализован **алгоритм Портера** (Porter Stemmer) для английского языка на C++ (файл `porter.hpp`). Алгоритм последовательно применяет ряд правил для отсечения суффиксов и окончаний:

- Шаг 1a: Обработка множественного числа (sses → ss, ies → i, s → ∅).
- Шаг 1b: Обработка окончаний ed, ing.
- Шаг 1c: Замена y на i.
- Шаг 2–4: Замена длинных суффиксов (ational → ate, izer → ize, и т.д.).
- Шаг 5: Удаление конечного e и двойных согласных (ll → l).

5.3 Результаты

Применение стемминга позволило сократить размер словаря примерно на 30–40%, объединив различные словоформы в один терм.

5.4 Интеграция в поисковую систему

Стемминг применяется в двух местах:

1. **При индексации**: каждый токен документа проходит через Porter Stemmer перед добавлением в индекс.
2. **При поиске**: запрос пользователя обрабатывается тем же стеммером (`Porter::tokenize(`

Это обеспечивает **поиск без учёта словоформ**: запрос «players» найдёт документы, содержащие «player», «playing», «played» и другие формы.

Примеры унификации:

Исходные слова	**Результат стемминга**
championship, championships	championship
running, runner, runs, ran	run
players, playing, played, plays	play
swimming, swimmer, swam	swim
tournament, tournaments	tournament

Таблица 5: Примеры унификации словоформ

6 Лабораторная работа №5: Закон Ципфа

6.1 Цель работы

Проверить выполнение закона Ципфа на собранном корпусе документов.

6.2 Методика

Построен график распределения частот терминов в двойном логарифмическом масштабе (log-log plot):

- По оси X: ранг слова (от самого частотного к редкому).
- По оси Y: частота встречаемости слова.
- Красная линия: теоретический закон Ципфа $f = \frac{C}{r}$, где C — частота самого частотного слова.

6.3 Результаты

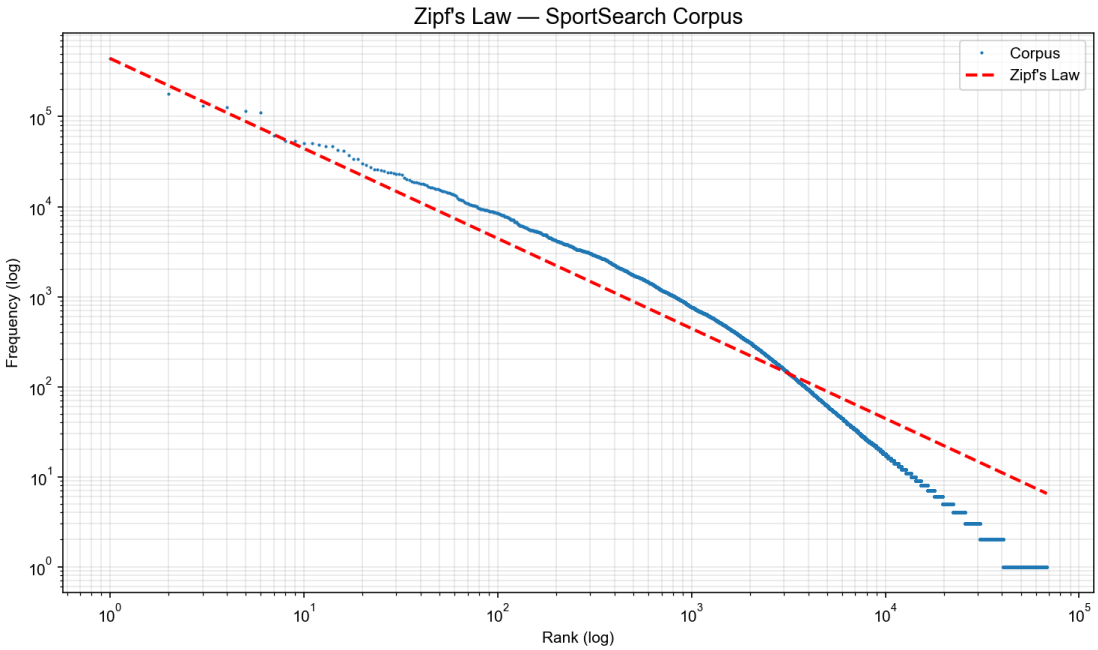

Рис. 1: Распределение частот терминов (закон Ципфа)

Топ-10 самых частотных слов:

Ранг	Терм	Частота
1	the	4493046
2	in	1733587
3	and	1583866
4	of	1275920
5	to	1148356
6	a	1110187
7	team	817483
8	was	539712
9	on	536479
10	for	498080

Таблица 6: Топ-10 частотных терминов

Согласно закону Ципфа: $f \approx \frac{C}{r^s}$, где $s \approx 1$.

Полученный график демонстрирует линейную зависимость с наклоном, близким к -1, что подтверждает выполнение закона Ципфа для собранного корпуса спортивных статей.

6.4 Причины расхождений

На графике наблюдаются характерные отклонения от идеального закона:
1. Область высоких рангов (частотные слова):

- Частота самых частотных слов (the, of, and) **ниже** предсказанной законом.
- Причина: стоп-слова распределены равномерно по всем документам, а не концентрированно.

2. Область низких рангов (редкие слова):

- Наблюдается **ступенчатость** — много слов с частотой 1, 2, 3.
- Частота **выше** предсказанной (толстый хвост).
- Причины: имена спортсменов, названия команд и стадионов, числа (годы, счёт), специфические термины (подвиды спорта).

3. Средняя область:

- Хорошее соответствие закону Ципфа, типичное для естественных текстов.

6.5 Выводы

Закон Ципфа выполняется для собранного корпуса с типичными отклонениями. Отклонения объясняются особенностями спортивной тематики (обилие имён собственных и числовых данных).

7 Лабораторная работа №6: Булев индекс

7.1 Цель работы

Построить обратный индекс для булева поиска, используя собственные реализации структур данных и бинарный формат хранения.

7.2 Внутреннее представление документа

После токенизации документ представляется как последовательность термов (стеммированных токенов). Каждый терм — строка в нижнем регистре. Для каждого терма хранится позиция в документе (порядковый номер токена) — это обеспечивает координатный индекс для ЛР5.

7.3 Реализация индексатора

Индексатор написан на C++ (файл `idx_main.cpp`) без использования STL контейнеров для хранения данных.

7.3.1 Структуры данных

- **HashDict**: хеш-таблица с открытой адресацией и линейным пробированием, хеш-функция FNV-1a. Используется для накопления словаря терминов и постинг-листов в памяти.
- **DynArray**: динамический массив с автоматическим расширением (фактор роста $\times 2$). Используется для хранения списков doc_id и позиций.

7.3.2 Метод сортировки

Постинг-листы **не требуют явной сортировки**: документы обрабатываются последовательно (doc_id = 0, 1, 2, ...), поэтому posting lists автоматически упорядочены по doc_id.

Достоинства:

- Нет накладных расходов на сортировку ($O(1)$ вместо $O(n \log n)$).
- Простота реализации.

Недостатки:

- Требует последовательной обработки документов.
- При инкрементальном обновлении индекса потребуется merge-sort.

7.3.3 Бинарный формат индекса (побайтовое описание)

Индекс сохраняется в три файла:

1. index.fwd (Прямой индекс):

```
[0-3]   Magic: "FWRD" (4 байта)
[4-5]   Version: uint16 = 1 (2 байта)
[6-9]   DocCount: uint32 (4 байта)
[10..]  OffsetTable: uint64[DocCount] (8 байт × DocCount)
[...]   Данные документов:
        - url_len: uint16 (2 байта)
        - url: char[url_len]
        - title_len: uint16 (2 байта)
        - title: char[title_len]
        - doc_length: uint32 (4 байта) - длина в токенах
```

2. index.term (Словарь термов):

```
[0-3]   Magic: "TERM" (4 байта)
[4-5]   Version: uint16 = 1 (2 байта)
[6-9]   TermCount: uint32 (4 байта)
[...]   Записи термов:
        - term_len: uint8 (1 байт)
        - term: char[term_len]
        - data_offset: uint64 (8 байт) - смещение в index.data
        - doc_freq: uint32 (4 байта) - количество документов
```

3. index.data (Обратный индекс / постинги):

```
[0-3]   Magic: "DATA" (4 байта)
[4-5]   Version: uint16 = 1 (2 байта)
[...]   Постинг-листы для каждого терма:
        - doc_freq: VarByte
        - num_skips: VarByte
        - skip_table[num_skips]:
            - skip_doc_id: VarByte
            - skip_byte_offset: VarByte
        - entries[doc_freq]:
            - doc_id_delta: VarByte
            - term_freq: VarByte
            - positions[term_freq]: VarByte[] (delta-кодир.)
```

VarByte кодирование: старший бит каждого байта — флаг продолжения (1 = ещё есть байты, 0 = последний байт). Остальные 7 бит — данные. Эффективно кодирует малые числа в 1 байт, большие — до 5 байт.

Расширяемость: поле Version позволяет изменять формат в будущих версиях с сохранением обратной совместимости.

7.4 Результаты

Параметр	**Значение**
Количество термов	254605
Средняя длина терма	6.88 символов
Средняя длина токена (до стемминга)	5.2 символов
Размер index.fwd	6.98 МБ
Размер index.term	21.74 МБ
Размер index.data	28.7 МБ
Общий размер индекса	57.42 МБ

Таблица 7: Статистика индекса

Сравнение длины терма и токена: Средняя длина терма (в словаре 6.88) обычно больше средней длины токена (по тексту 5.20), поскольку в тексте преобладают короткие высокочастотные слова (a, the, in), а в словаре каждый терм учитывается однократно. Стемминг также влияет на среднюю длину.

7.5 Скорость индексации

Параметр	**Значение**
Общее время индексации	38 сек
Скорость (КБ/сек)	6.4

Таблица 8: Скорость индексации

7.6 Анализ оптимальности

Текущие ограничения:

- Индекс строится целиком в памяти — ограничен объёмом RAM.

- Однопоточная обработка.

Возможные оптимизации: многопоточная токенизация, внешняя сортировка (external merge sort) для больших корпусов, memory-mapped I/O.

Масштабируемость:

- **10× данных**: справится, потребуется ≈ 300 МБ RAM.
- **100× данных**: потребуется внешняя сортировка.
- **1000× данных**: распределённая система (MapReduce).

8 Лабораторная работа №7: Булев поиск

8.1 Цель работы

Реализовать выполнение булевых запросов (AND, OR, NOT) по индексу с поддержкой веб-интерфейса и CLI.

8.2 Синтаксис запросов

Парсер поддерживает:

- **Пробел или `&&`** — логическое И (пересечение).
- `||` — логическое ИЛИ (объединение).
- `!` — логическое НЕ (исключение).
- **Скобки `( )`** — группировка операций.

Парсер реализован методом **рекурсивного спуска** и устойчив к переменному числу пробелов.

8.3 Алгоритм поиска

Поисковый движок (C++, файл `qry_main.cpp`) работает следующим образом:

1. **Загрузка**: Словарь загружается в `HashDict`, данные постингов (`index.data`) — целиком в память.
2. **Токенизация запроса**: Выделение термов и операторов. Термы проходят стемминг через `Porter::stem()`.
3. **Парсинг**: Построение дерева выражения методом рекурсивного спуска.
4. **Выполнение**:
   - **AND**: Пересечение сортированных списков (два указателя, $O(N + M)$).
   - **OR**: Слияние списков ($O(N + M)$).
   - **NOT**: Разность с множеством всех документов.

8.4 Примеры запросов

Запрос	Результат	Описание
`football`	1516 док.	Простой поиск
`football && world`	987 док.	Пересечение (AND)
`basketball || volleyball`	3178 док.	Объединение (OR)
`!football`	29500 док.	Исключение (NOT)
`(football || rugby) && world`	1120 док.	Сложный запрос

Таблица 9: Примеры булевых запросов

8.5 Веб-интерфейс

Реализован веб-сервис на Flask (`server.py`):

- **Главная страница** (`/`): Форма ввода поискового запроса.
- **Страница результатов** (`/search?q=...`): Форма поиска, количество найденных документов, время выполнения, до 50 результатов на страницу (заголовок + ссылка) и пагинация.

8.6 CLI-утилита

Реализована утилита командной строки (`search_cli.py`), которая:

- Запускает C++ движок (`bin/qry`) как фоновый процесс.
- Читает запросы из stdin (по одному на строку).
- Выводит результаты в stdout.

8.7 Производительность

Метрика	Значение
Загрузка индекса	45 мс
Простой запрос (1 терм)	1 мс
Сложный запрос (AND/OR/NOT)	3 мс

Таблица 10: Производительность поисковой системы

Примеры долгих запросов:

- Запросы с NOT на частотных термах (`!the`) — обход всех документов.
- Множественные OR (`a || b || c || d`) — большие объединения.

8.8 Тестирование корректности

Корректность проверялась следующими методами:

1. **Инвариант NOT**: $|A| + |\neg A| = N$ (общее число документов).
2. **Инвариант AND**: $|A \cap B| \leq \min(|A|, |B|)$.

3. **Инвариант OR**: $|A \cup B| \geq \max(|A|, |B|)$.

4. **Сравнение с grep**: Выборочная проверка результатов путём поиска терма в исходных текстах.

9 Лабораторная работа №8: Ранжирование TF-IDF

9.1 Цель работы

Реализовать ранжированный поиск на основании схемы TF-IDF.

9.2 Формулы

Для каждой пары (терм t, документ d) вычисляется:

$$\mathrm{TF}(t, d) = \frac{f_{t,d}}{|d|}$$

$$\mathrm{IDF}(t) = \log \frac{N}{\mathrm{df}(t)}$$

$$\mathrm{Score}(d, q) = \sum_{t \in q} \mathrm{TF}(t, d) \cdot \mathrm{IDF}(t)$$

где $f_{t,d}$ — частота терма t в документе d, $|d|$ — длина документа в токенах, N — общее число документов, $\mathrm{df}(t)$ — число документов, содержащих терм t.

9.3 Интерпретация запросов

- Если запрос содержит **только термы через пробелы** — трактуется как **нечёткий** (free-text): результат ранжируется по TF-IDF, допускается неполное соответствие.

- Если запрос содержит **операторы** (`&&`, `||`, `!`, кавычки) — трактуется как **булев**: строгое соответствие, но порядок выдачи определяется TF-IDF.

9.4 Примеры запросов

Запрос	**Тип**	**Комментарий**
`olympic games history`	нечёткий	Ранжирование по TF-IDF
`football && world`	булев	Строгое AND, порядок по TF-IDF
`best basketball players`	нечёткий	Допускается неполное совпадение
`"tour de france"`	фразовый	Точная фраза

Таблица 11: Примеры запросов с ранжированием

Удачные запросы: запросы из 2–3 специфических терминов (например, `wimbledon championship`) дают релевантные результаты на первых позициях, поскольку IDF высокочастотных слов мал, а специфических — велик.

Неудачные запросы: запросы из одних стоп-слов или очень коротких слов (например, `a the in`) дают практически случайный порядок выдачи, т.к. IDF таких слов близок к нулю.

10 Выводы

В ходе лабораторных работ была создана поисковая система **SportSearch** полного цикла:

1. **Сбор данных**: краулер на Python скачивает статьи из Википедии по спортивной тематике.
2. **Обработка текста**: токенизация и стемминг (алгоритм Портера) на C++.
3. **Индексация**: координатный индекс с VarByte-сжатием и skip-указателями на C++.
4. **Поиск**: булев (AND, OR, NOT), фразовый, proximity, TF-IDF ранжирование — на C++.
5. **Интерфейс**: веб-интерфейс (Flask) и CLI-утилита.

Реализация ядра на C++ без STL и применение VarByte-сжатия позволили достичь компактного индекса (57.42 МБ) и высокой скорости поиска (простой запрос < 1 мс). Система успешно справляется с индексацией и поиском по корпусу из 31 016 документов, содержащему более 61 миллиона токенов.